OS IA Assignment Activity - 2 - AI-Assisted Dashboard Development and Deadlock Analysis

1. Use of AI and Generative AI Tools

Students (in a group of 4/5) are encouraged to use AI-assisted development tools to support the design, implementation, testing, and documentation of the Deadlock Analysis and Resource Allocation Dashboard.

Suggested AI Tools

Students may use any suitable tool, including:

- **ChatGPT / Google Gemini / Claude:** To understand deadlock concepts, generate algorithm pseudocode, develop test cases, review code, and explain Banker's Algorithm.
- **Lovable / Bolt.new:** To generate dashboard interfaces and interactive web application prototypes.
- **GitHub Copilot / Cursor:** To assist with code completion, debugging, and refactoring.
- **Streamlit:** To implement interactive dashboards using Python.
- **Canva:** To prepare diagrams, presentation slides, and visual explanations of deadlock conditions.

Students may combine multiple tools or use another instructor-approved platform.

Guidelines for Responsible AI Usage

1. Students may use AI tools to generate initial code, UI layouts, pseudocode, test data, and explanations.
2. AI-generated code must be reviewed, tested, and modified by the students as required.
3. Students must independently verify the correctness of the Banker's Algorithm, safe-sequence calculation, and resource-request decisions.
4. Students must not assume that AI-generated explanations or outputs are correct without testing them.
5. Each group must mention the AI tools used and briefly describe their contribution.
6. Students must be able to explain and modify their implementation during the demonstration or viva without depending entirely on AI-generated output.

7. Do not upload confidential institutional information, credentials, or sensitive data to external AI tools.

AI Usage Record

Each group must document:

AI Tool Used	**Purpose**	**Output Generated**	**Student Verification**
_____	Dashboard design	_____	_____
_____	Algorithm/code assistance	_____	_____
_____	Test-case generation or debugging	_____	_____

2. Analytical Evaluation of Deadlock Test Cases

In addition to demonstrating the dashboard, students must analyze the behavior of the system under different resource-allocation conditions. The analysis must explain *why* a request is granted, delayed, rejected, or identified as unsafe.

Required Analysis Scenarios

Scenario A: Safe State

Configure the processes, resource instances, Allocation matrix, Maximum matrix, and Available vector so that the Banker's Algorithm finds a safe sequence.

Students must:

- Calculate the Need matrix.
- Identify the process that can complete with the currently available resources.
- Update the Work vector after each process completes.
- Display the complete safe sequence.
- Explain why the state is safe.

Scenario B: Request Exceeds Available Resources

Submit a valid request that does not exceed the process's remaining Need but exceeds the currently Available resources.

Students must explain why the request cannot be granted immediately and verify that the current allocation remains unchanged.

Scenario C: Request Produces an Unsafe State

Submit a request that satisfies both the process's remaining Need and the currently Available vector, but would make the system unsafe if granted.

Students must:

- Temporarily simulate the allocation.
- Run the safety algorithm.
- Show that no complete safe sequence exists.
- Deny the request and restore the original state.
- Explain why an unsafe state is not necessarily a deadlocked state.

Scenario D: Deadlock Detection

Construct a separate resource-allocation scenario in which processes are waiting for resources held by one another.

Students must:

- Identify the relevant resource-allocation relationships.
- Demonstrate a circular wait or a suitable deadlock-detection result.
- Explain the distinction between a potential circular wait and confirmed deadlock under the stated resource assumptions.
- Demonstrate one recovery action, such as terminating a process and releasing its resources.

Scenario E: Deadlock Prevention

Modify the resource-allocation policy to prevent at least one necessary condition of deadlock.

Examples include imposing a fixed resource-acquisition order to prevent circular wait or requiring processes to request all needed resources before execution to eliminate hold and wait.

Students must compare the original and modified scenarios and discuss the trade-offs, such as lower resource utilization or reduced concurrency.

Scenario F: Boundary and Invalid Inputs

Test zero available resources, a process with zero remaining Need, invalid process identifiers, negative requests, and allocations exceeding declared maximum demands.

Students must distinguish valid boundary cases from invalid input and explain how the dashboard handles each case.

3. Required Analytical Results Table

Complete the following table using actual dashboard outputs.

Test Case	**System Condition**	**Actual Result**	**Expected Result**	**Analysis / Conclusion**
TC1	Safe state	_____	Safe sequence found	_____
TC2	Request exceeds Available	_____	Request not granted immediately	_____
TC3	Request makes state unsafe	_____	Request denied; state unchanged	_____
TC4	Deadlocked scenario	_____	Deadlock identified by the selected detection method	_____
TC5	Prevention policy applied	_____	Selected deadlock condition eliminated	_____
TC6	Boundary and invalid inputs	_____	Correct handling or validation message	_____

The dashboard must distinguish between **safe, unsafe, deadlocked, and invalid-input states** wherever applicable. An unsafe state must not automatically be labelled as

deadlock, and an unavailable request must not automatically be treated as an invalid request.

4. Additional Assessment Expectations

The original total remains **20 marks**. AI usage is not a separate bonus category; it is evaluated within the existing criteria.

Criterion	Marks	Additional Expectation
Dashboard design with Deadlock concepts and handling strategies	5	Correct explanation of prevention, avoidance, detection, and recovery. Interactive, clear, and appropriately visualized interface
Banker's Algorithm implementation	5	Correct Need calculation, safety check, request validation, and safe sequence
Test coverage and analytical evaluation	5	Actual results, reasoning, edge cases, and distinction between unsafe and deadlocked states
Demonstration, individual understanding, and Q&A	5	Explain the code, validate AI-generated contributions, and justify decisions
Total	**20**	

• All reported actual results must come from testing the group's own dashboard. Students must verify AI-generated code and outputs before including them in the submission.

• Groups may use a common instructor-provided sample for an initial Banker's Algorithm check, if supplied. Each group must also create and explain its own test cases for the remaining required scenarios.

A. Minimum Dashboard Requirements

To ensure a consistent understanding of the expected work, all groups must follow the clarifications below.

B. Resource Requests: Waiting, Denial, and State Restoration

- Display clear status messages for safe, unsafe, waiting/deferred request, detected deadlock, and invalid input. Do not treat these states as interchangeable.
- Provide separate functions or views for deadlock avoidance, deadlock detection, and deadlock prevention/recovery.
- Validate resource requests and show whether a request can be granted safely.
- Implement Banker's safety algorithm and display the Work vector updates and complete safe sequence when one exists.
- Calculate the Need matrix automatically using Need = Maximum − Allocation.
- Accept or configure the number of processes, resource types, Allocation matrix, Maximum matrix, and Available vector.

C. Deadlock Detection Assumptions

- An unsafe state is not necessarily a deadlocked state. Explain this distinction in the dashboard and during the demonstration.
- If a request is within both Need and Available but would result in an unsafe state, the Banker's avoidance procedure must deny the request. Restore and retain the original allocation state.
- If a request is within the process's remaining Need but exceeds Available resources, report that it cannot be granted immediately (waiting/deferred). Do not automatically label it invalid or permanently denied.

D. Individual Contribution and Viva

- For the recovery demonstration, explain which process is terminated or otherwise recovered, which resources are released, and how this allows progress.
- Use a detection method appropriate to the stated resource model. A cycle in a resource-allocation graph is not, by itself, conclusive proof of deadlock in every multiple-instance-resource scenario.
- Clearly state whether the deadlock-detection example uses single-instance or multiple-instance resource types.

E. Submission Requirements

- During the demonstration or viva, each student may be asked to explain an algorithm step, justify a test result, or make a small code/configuration change.
- Every group member must understand the complete solution, not only the part assigned to them.

F. Test Data and Originality

- The instructor will communicate the submission deadline and submission location separately.
- Mention the development tools/platforms used. Do not upload confidential institutional information, credentials, or sensitive data to external AI tools.
- Include screenshots or other evidence of actual outputs for the required scenarios, where appropriate.
- Submit the working dashboard, complete source code, completed analytical test-case table, and AI Usage Record.

5. Additional Implementation and Submission Clarifications

Final Instruction

Every group must submit a working dashboard, complete source code, the completed test-case analysis, and an AI Usage Record. During the six-minute demonstration, each group must show at least one safe state, one resource request that is not granted (explaining whether it is waiting or denied), and one deadlock-related scenario. All four/five students must understand the complete solution and be prepared to explain the correctness of the results.

Particular weight will be given to the student with clear conceptual idea for distinction between deadlock avoidance and deadlock detection. The Banker's Algorithm checks whether granting a request preserves a safe state; it does not, by itself, detect an existing deadlock. Students should demonstrate these as separate functions in their dashboard.